

Computer Vision Interview Questions & Answers Cheat Sheet

Table of Contents

- 1. [Computer Vision Fundamentals](#)
- 2. [Image Processing Basics](#)
- 3. [Convolutional Neural Networks \(CNNs\)](#)
- 4. [Popular CNN Architectures](#)
- 5. [Object Detection](#)
- 6. [Image Segmentation](#)
- 7. [Feature Detection & Matching](#)
- 8. [Transfer Learning in CV](#)
- 9. [Generative Models \(GANs, VAEs\)](#)
- 10. [OpenCV & Implementation](#)
- 11. [Evaluation Metrics](#)
- 12. [Advanced Topics](#)
- 13. [Practical Applications](#)
- 14. [Deployment & Optimization](#)

Computer Vision Fundamentals

Q1: What is Computer Vision and how does it differ from Image Processing?

Answer:

Computer Vision: Field of AI that enables machines to interpret and understand visual information from images/videos to make decisions.

Image Processing: Focuses on manipulating and enhancing images through mathematical operations.

Aspect	Computer Vision	Image Processing
Goal	Extract high-level understanding	Enhance/modify image quality
Output	Decisions, classifications, detections	Modified/enhanced images
Intelligence	AI-driven interpretation	Rule-based transformations

Aspect	Computer Vision	Image Processing
Examples	Face recognition, object detection	Noise reduction, histogram equalization

Computer Vision Pipeline:

Raw Image → Preprocessing → Feature Extraction → Pattern Recognition → Decision Making

Q2: What are the main challenges in Computer Vision?

Answer:

1. Illumination Variations:

- Different lighting conditions affect appearance
- Shadow and reflection issues
- Solutions: Histogram equalization, adaptive methods

2. Scale and Rotation Invariance:

- Objects appear at different sizes and orientations
- Solutions: Data augmentation, multi-scale features

3. Occlusion:

- Objects partially hidden by other objects
- Solutions: Part-based models, attention mechanisms

4. Intra-class Variation:

- Same object class can look very different
- Solutions: Large datasets, data augmentation

5. Inter-class Similarity:

- Different classes may look similar
- Solutions: Better feature representations, fine-grained classification

6. Real-time Processing:

- Need for fast inference
- Solutions: Model optimization, efficient architectures

Q3: What is a digital image and how is it represented?

Answer:

Digital Image: 2D array of pixels where each pixel represents intensity/color values.

Representations:

1. Grayscale Image:

- Single channel: $I(x,y) \in [0, 255]$
- Shape: (height, width)

2. RGB Color Image:

- Three channels: R, G, B
- Shape: (height, width, 3)
- Each channel: $[0, 255]$

3. Other Color Spaces:

- **HSV**: Hue, Saturation, Value
- **LAB**: Lightness, A (green-red), B (blue-yellow)
- **YUV**: Luminance, Chrominance

Example:

```
import cv2
import numpy as np

# Load image
img = cv2.imread('image.jpg')
print(f"Shape: {img.shape}") # (height, width, channels)
print(f"Data type: {img.dtype}") # uint8
print(f"Pixel range: {img.min()} to {img.max()}") # 0 to 255

# Convert color spaces
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

Image Processing Basics

Q4: Explain different types of image filters and their applications.

Answer:

1. Smoothing Filters:

Gaussian Blur:

```
# Removes noise, reduces detail
blurred = cv2.GaussianBlur(img, (15, 15), 0)
# Kernel size should be odd
```

Median Filter:

```
# Removes salt-and-pepper noise
median = cv2.medianBlur(img, 5)
# Preserves edges better than Gaussian
```

2. Sharpening Filters:

```
# Enhance edges and details
kernel = np.array([[ -1, -1, -1],
                   [ -1,  9, -1],
                   [ -1, -1, -1]])
sharpened = cv2.filter2D(img, -1, kernel)
```

3. Edge Detection Filters:

Sobel Operator:

```
# Gradient-based edge detection
sobelx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel = np.sqrt(sobelx**2 + sobely**2)
```

Canny Edge Detection:

```
# Multi-step edge detection
edges = cv2.Canny(gray, threshold1=50, threshold2=150)
# More accurate than Sobel
```

Applications:

- **Smoothing:** Preprocessing, noise reduction
- **Sharpening:** Image enhancement, detail improvement
- **Edge Detection:** Object detection, image segmentation

Q5: What is histogram equalization and when is it useful?

Answer:

Histogram Equalization: Technique to improve image contrast by redistributing pixel intensities.

Process:

1. Calculate histogram of image
2. Compute cumulative distribution function (CDF)
3. Normalize CDF to [0, 255] range
4. Map original pixels to new values

Implementation:

```

# Global histogram equalization
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
equalized = cv2.equalizeHist(gray)

# Adaptive histogram equalization (CLAHE)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
adaptive_eq = clahe.apply(gray)

# Compare histograms
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.hist(gray.ravel(), 256, [0, 256])
plt.title('Original')

plt.subplot(1, 3, 2)
plt.hist(equalized.ravel(), 256, [0, 256])
plt.title('Global Equalization')

plt.subplot(1, 3, 3)
plt.hist(adaptive_eq.ravel(), 256, [0, 256])
plt.title('Adaptive Equalization')

```

When to Use:

- **Low contrast images:** Medical imaging, underwater images
- **Preprocessing:** Before feature extraction
- **Avoid when:** Images already have good contrast (can over-enhance)

CLAHE vs Global:

- **CLAHE:** Prevents over-amplification of noise, works on tiles
- **Global:** Simple but can amplify noise in uniform regions

Convolutional Neural Networks (CNNs)

Q6: Explain the architecture and working of CNNs.

Answer:

CNN Architecture: Neural network designed for grid-like data (images) using convolution operations.

Key Components:

1. Convolutional Layer:

```

# Mathematical operation

```

```
output[i,j] =  $\sum \sum \text{input}[i+m, j+n] \times \text{kernel}[m, n] + \text{bias}$ 
```

Properties:

- **Parameter Sharing:** Same filter applied across image
- **Translation Invariance:** Detects features regardless of position
- **Local Connectivity:** Each neuron connected to local region

2. Pooling Layer:

```
# Max pooling
max_pool = tf.keras.layers.MaxPooling2D(pool_size=(2, 2))

# Average pooling
avg_pool = tf.keras.layers.AveragePooling2D(pool_size=(2, 2))
```

Purpose:

- Reduce spatial dimensions
- Provide translation invariance
- Reduce computational cost

3. Fully Connected Layer:

- Traditional neural network layer
- Used for final classification
- Requires flattened input

Complete CNN Example:

```
import tensorflow as tf

model = tf.keras.Sequential([
    # Convolutional blocks
    tf.keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(224, 224, 3)),
    tf.keras.layers.MaxPooling2D((2, 2)),

    tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2, 2)),

    tf.keras.layers.Conv2D(128, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2, 2)),

    # Classification head
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(num_classes, activation='softmax')
])
```

Q7: What is the difference between valid and same padding?

Answer:

Padding: Adding pixels around input to control output size.

Valid Padding (No Padding):

- Output size: $(n - f + 1) \times (n - f + 1)$
- Shrinks feature maps
- May lose edge information

Same Padding:

- Output size equals input size
- Preserves spatial dimensions
- Adds zeros around border

Calculations:

```
def calculate_output_size(input_size, kernel_size, stride, padding):
    if padding == 'valid':
        return (input_size - kernel_size) // stride + 1
    elif padding == 'same':
        return input_size // stride

# Example
input_size = 32
kernel_size = 5
stride = 1

valid_output = calculate_output_size(32, 5, 1, 'valid') # 28
same_output = calculate_output_size(32, 5, 1, 'same')   # 32
```

Implementation:

```
# Valid padding
conv_valid = tf.keras.layers.Conv2D(64, (3, 3), padding='valid')

# Same padding
conv_same = tf.keras.layers.Conv2D(64, (3, 3), padding='same')

# Custom padding
padded_input = tf.pad(input_tensor, [[0,0], [1,1], [1,1], [0,0]], "CONSTANT")
```

When to Use:

- **Valid:** When you want to progressively reduce spatial size
- **Same:** When you want to preserve spatial dimensions
- **Typical Pattern:** Use 'same' padding with stride=1, 'valid' with pooling

Popular CNN Architectures

Q8: Compare and contrast LeNet, AlexNet, VGG, and ResNet architectures.

Answer:

Architecture	Year	Key Innovation	Depth	Parameters
LeNet-5	1998	First CNN for digit recognition	7 layers	~60K
AlexNet	2012	ReLU, Dropout, GPU training	8 layers	~60M
VGG-16	2014	Small 3×3 filters, deeper networks	16 layers	~138M
ResNet-50	2015	Skip connections, very deep	50 layers	~25M

Detailed Comparison:

1. LeNet-5 (1998):

```
# Simple architecture for MNIST
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(6, (5, 5), activation='tanh', input_shape=(32, 32, 1)),
    tf.keras.layers.AveragePooling2D((2, 2)),
    tf.keras.layers.Conv2D(16, (5, 5), activation='tanh'),
    tf.keras.layers.AveragePooling2D((2, 2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(120, activation='tanh'),
    tf.keras.layers.Dense(84, activation='tanh'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

2. AlexNet (2012):

- **Innovations:** ReLU activation, Dropout, Data augmentation
- **Architecture:** 5 Conv + 3 FC layers
- **Performance:** Won ImageNet 2012

3. VGG-16 (2014):

```
# Key insight: Stack of 3x3 conv layers
def vgg_block(num_filters, num_convs):
    block = tf.keras.Sequential()
    for _ in range(num_convs):
        block.add(tf.keras.layers.Conv2D(num_filters, (3, 3),
                                          activation='relu', padding='same'))
    block.add(tf.keras.layers.MaxPooling2D((2, 2)))
    return block
```

4. ResNet (2015):

```
# Skip connections solve vanishing gradient
def residual_block(x, filters, stride=1):
    shortcut = x

    # Main path
    x = tf.keras.layers.Conv2D(filters, (3, 3), strides=stride, padding='same')(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.ReLU()(x)

    x = tf.keras.layers.Conv2D(filters, (3, 3), padding='same')(x)
    x = tf.keras.layers.BatchNormalization()(x)

    # Adjust shortcut if needed
    if stride != 1:
        shortcut = tf.keras.layers.Conv2D(filters, (1, 1), strides=stride)(shortcut)
        shortcut = tf.keras.layers.BatchNormalization()(shortcut)

    # Add skip connection
    x = tf.keras.layers.Add()([x, shortcut])
    x = tf.keras.layers.ReLU()(x)
    return x
```

Q9: What are skip connections in ResNet and why are they important?

Answer:

Skip Connections: Direct paths that allow gradients to flow from later layers to earlier layers.

Problem Solved:

- **Vanishing Gradient:** Gradients become too small in deep networks
- **Degradation Problem:** Deeper networks perform worse than shallow ones

Mathematical Formulation:

Traditional: $H(x) = F(x)$

ResNet: $H(x) = F(x) + x$

Where $F(x)$ is the residual mapping

Implementation:

```
def residual_block(input_tensor, filters):
    # Main path
    x = tf.keras.layers.Conv2D(filters, (3, 3), padding='same')(input_tensor)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.ReLU()(x)

    x = tf.keras.layers.Conv2D(filters, (3, 3), padding='same')(x)
    x = tf.keras.layers.BatchNormalization()(x)

    # Skip connection
```

```

output = tf.keras.layers.Add()([x, input_tensor])
output = tf.keras.layers.ReLU()(output)

return output

```

Benefits:

1. **Gradient Flow:** Direct path for gradients to earlier layers
2. **Identity Mapping:** Network can learn identity function easily
3. **Training Stability:** More stable training of very deep networks
4. **Better Performance:** Enables training of 100+ layer networks

Variants:

- **Identity Shortcut:** Direct connection when dimensions match
- **Projection Shortcut:** 1×1 conv when dimensions don't match
- **Bottleneck Design:** 1×1 → 3×3 → 1×1 conv pattern

Object Detection

Q10: Explain the evolution from R-CNN to Faster R-CNN.

Answer:

Object Detection Evolution:

1. R-CNN (2014):

```

# Multi-stage approach
def r_cnn_process(image):
    # 1. Generate ~2000 region proposals using selective search
    proposals = selective_search(image)

    # 2. Extract features for each proposal using CNN
    features = []
    for proposal in proposals:
        cropped = crop_and_resize(image, proposal)
        feature = cnn.extract_features(cropped)
        features.append(feature)

    # 3. Classify each proposal using SVM
    # 4. Refine bounding boxes using regression
    return predictions

```

Problems: Very slow (47s per image), multiple stages

2. Fast R-CNN (2015):

```

# Improved efficiency
def fast_r_cnn_process(image):
    # 1. Generate feature map for entire image
    feature_map = cnn.forward(image)

    # 2. Generate region proposals
    proposals = selective_search(image)

    # 3. ROI pooling to extract fixed-size features
    for proposal in proposals:
        roi_features = roi_pooling(feature_map, proposal)
        classification = classifier(roi_features)
        bbox_regression = regressor(roi_features)

    return predictions

```

Improvements: 9x faster training, 213x faster testing

3. Faster R-CNN (2015):

```

# End-to-end trainable
class FasterRCNN:
    def __init__(self):
        self.backbone = ResNet50() # Feature extractor
        self.rpn = RegionProposalNetwork() # Replace selective search
        self.roi_head = ROIHead() # Classification and regression

    def forward(self, image):
        # 1. Extract features
        feature_maps = self.backbone(image)

        # 2. Generate proposals using RPN
        proposals, rpn_losses = self.rpn(feature_maps)

        # 3. ROI pooling and final predictions
        predictions = self.roi_head(feature_maps, proposals)

        return predictions, rpn_losses

```

Key Innovation: Region Proposal Network (RPN) replaces selective search

Q11: How does YOLO (You Only Look Once) work?

Answer:

YOLO Approach: Treats object detection as single regression problem.

Key Concept:

1. Divide image into $S \times S$ grid
2. Each grid cell predicts B bounding boxes
3. Each bounding box has confidence score

4. Each grid cell predicts class probabilities

Architecture:

```
class YOLOv1(tf.keras.Model):
    def __init__(self, S=7, B=2, C=20):
        super().__init__()
        self.S = S # Grid size
        self.B = B # Boxes per cell
        self.C = C # Number of classes

        # Backbone (modified GoogLeNet)
        self.backbone = self.build_backbone()

        # Detection head
        self.detection_head = tf.keras.Sequential([
            tf.keras.layers.Dense(4096, activation='relu'),
            tf.keras.layers.Dropout(0.5),
            tf.keras.layers.Dense(S * S * (B * 5 + C))
        ])

    def forward(self, x):
        features = self.backbone(x)
        predictions = self.detection_head(features)

        # Reshape to (batch, S, S, B*5 + C)
        predictions = tf.reshape(predictions, (-1, self.S, self.S, self.B * 5 + self.C))

        return predictions
```

Output Format:

- Each grid cell: (x, y, w, h, confidence) × B + class_probs × C
- x, y: Center coordinates relative to cell
- w, h: Width, height relative to image
- confidence: P(Object) × IOU

Loss Function:

```
def yolo_loss(predictions, targets, lambda_coord=5, lambda_noobj=0.5):
    # Coordinate loss
    coord_loss = lambda_coord * tf.reduce_sum(
        tf.square(pred_x - true_x) + tf.square(pred_y - true_y)
    )

    # Size loss (square root to reduce impact of large boxes)
    size_loss = lambda_coord * tf.reduce_sum(
        tf.square(tf.sqrt(pred_w) - tf.sqrt(true_w)) +
        tf.square(tf.sqrt(pred_h) - tf.sqrt(true_h))
    )

    # Confidence loss (object present)
    obj_loss = tf.reduce_sum(tf.square(pred_conf - true_conf))
```

```

# Confidence loss (no object)
noobj_loss = lambda_noobj * tf.reduce_sum(tf.square(pred_conf - 0))

# Classification loss
class_loss = tf.reduce_sum(tf.square(pred_class - true_class))

return coord_loss + size_loss + obj_loss + noobj_loss + class_loss

```

Advantages:

- **Speed:** Real-time detection (45 FPS)
- **Global Context:** Sees entire image
- **Unified Architecture:** End-to-end training

Limitations:

- **Small Objects:** Struggles with small/grouped objects
- **Aspect Ratios:** Limited to predefined aspect ratios
- **Accuracy:** Lower than two-stage methods

Q12: What is Non-Maximum Suppression (NMS)?

Answer:

Non-Maximum Suppression: Post-processing technique to eliminate redundant overlapping bounding boxes.

Problem: Object detectors often generate multiple detections for same object.

Algorithm:

```

def nms(boxes, scores, iou_threshold=0.5):
    """
    Args:
        boxes: Array of bounding boxes (N, 4) - [x1, y1, x2, y2]
        scores: Array of confidence scores (N,)
        iou_threshold: Overlap threshold for suppression

    Returns:
        kept_indices: Indices of boxes to keep
    """
    # Sort boxes by confidence score (descending)
    indices = np.argsort(scores)[::-1]

    kept_indices = []

    while len(indices) > 0:
        # Keep box with highest confidence
        current_idx = indices[0]
        kept_indices.append(current_idx)

```

```

        if len(indices) == 1:
            break

        # Calculate IoU with remaining boxes
        current_box = boxes[current_idx]
        remaining_boxes = boxes[indices[1:]]

        ious = calculate_iou(current_box, remaining_boxes)

        # Remove boxes with IoU > threshold
        indices = indices[1:][ious <= iou_threshold]

    return kept_indices

def calculate_iou(box1, boxes):
    """Calculate IoU between one box and multiple boxes"""
    # Calculate intersection area
    x1 = np.maximum(box1[0], boxes[:, 0])
    y1 = np.maximum(box1[1], boxes[:, 1])
    x2 = np.minimum(box1[2], boxes[:, 2])
    y2 = np.minimum(box1[3], boxes[:, 3])

    intersection = np.maximum(0, x2 - x1) * np.maximum(0, y2 - y1)

    # Calculate union area
    box1_area = (box1[2] - box1[0]) * (box1[3] - box1[1])
    boxes_area = (boxes[:, 2] - boxes[:, 0]) * (boxes[:, 3] - boxes[:, 1])
    union = box1_area + boxes_area - intersection

    return intersection / union

```

Soft-NMS Improvement:

```

def soft_nms(boxes, scores, sigma=0.5):
    """
    Soft-NMS: Gradually reduce scores instead of hard removal
    """
    N = len(boxes)
    kept_indices = []

    for i in range(N):
        max_idx = np.argmax(scores)
        if scores[max_idx] < threshold:
            break

        kept_indices.append(max_idx)

        # Calculate IoU with all other boxes
        ious = calculate_iou(boxes[max_idx], boxes)

        # Soft suppression: decay scores based on IoU
        scores = scores * np.exp(-(ious ** 2) / sigma)
        scores[max_idx] = -1 # Mark as processed

```

```
return kept_indices
```

Parameters:

- **IoU Threshold:** Typically 0.3-0.7
- **Higher Threshold:** Keeps more boxes (less suppression)
- **Lower Threshold:** More aggressive suppression

Image Segmentation

Q13: What are the different types of image segmentation?

Answer:

Image Segmentation Types:

1. Semantic Segmentation:

- **Goal:** Classify each pixel into predefined classes
- **Output:** Same class pixels have same label
- **Example:** All pixels belonging to "car" get same label

```
# Semantic segmentation with U-Net
def build_unet(input_shape, num_classes):
    inputs = tf.keras.layers.Input(input_shape)

    # Encoder (downsampling)
    conv1 = tf.keras.layers.Conv2D(64, 3, activation='relu', padding='same')(inputs)
    conv1 = tf.keras.layers.Conv2D(64, 3, activation='relu', padding='same')(conv1)
    pool1 = tf.keras.layers.MaxPooling2D(2)(conv1)

    # Decoder (upsampling)
    up1 = tf.keras.layers.UpSampling2D(2)(conv1)
    merge1 = tf.keras.layers.concatenate([conv1, up1])

    # Final layer
    outputs = tf.keras.layers.Conv2D(num_classes, 1, activation='softmax')(merge1)

    return tf.keras.Model(inputs, outputs)
```

2. Instance Segmentation:

- **Goal:** Distinguish between different instances of same class
- **Output:** Different instances have different labels
- **Example:** Two cars get different labels

```
# Mask R-CNN for instance segmentation
class MaskRCNN:
    def __init__(self):
        self.backbone = ResNet50FPN()
        self.rpn = RegionProposalNetwork()
        self.roi_head = ROIHead()
        self.mask_head = MaskHead() # Additional mask prediction

    def forward(self, images):
        features = self.backbone(images)
        proposals = self.rpn(features)

        # Bounding box and classification
        bbox_pred, cls_pred = self.roi_head(features, proposals)

        # Mask prediction for each instance
        masks = self.mask_head(features, proposals)

        return bbox_pred, cls_pred, masks
```

3. Panoptic Segmentation:

- **Goal:** Combines semantic and instance segmentation
- **Output:** Unified representation of scene understanding

Applications:

- **Medical Imaging:** Organ segmentation
- **Autonomous Driving:** Road scene understanding
- **Agriculture:** Crop monitoring

Q14: Explain the U-Net architecture and its applications.

Answer:

U-Net: Encoder-decoder architecture with skip connections, designed for biomedical image segmentation.

Architecture Components:

1. Encoder (Contracting Path):

- Repeated: Conv → Conv → MaxPool
- Captures context and reduces spatial resolution
- Increases feature channels

2. Decoder (Expansive Path):

- Repeated: Upsample → Concat → Conv → Conv
- Recovers spatial resolution

- Uses skip connections from encoder

3. Skip Connections:

- Copy features from encoder to decoder
- Preserves fine-grained details
- Enables precise localization

Implementation:

```
def unet_model(input_size=(256, 256, 1), num_classes=1):
    inputs = tf.keras.Input(input_size)

    # Encoder
    c1 = conv_block(inputs, 64)
    p1 = tf.keras.layers.MaxPooling2D(2)(c1)

    c2 = conv_block(p1, 128)
    p2 = tf.keras.layers.MaxPooling2D(2)(c2)

    c3 = conv_block(p2, 256)
    p3 = tf.keras.layers.MaxPooling2D(2)(c3)

    c4 = conv_block(p3, 512)
    p4 = tf.keras.layers.MaxPooling2D(2)(c4)

    # Bottleneck
    c5 = conv_block(p4, 1024)

    # Decoder
    u6 = tf.keras.layers.UpSampling2D(2)(c5)
    u6 = tf.keras.layers.concatenate([u6, c4])
    c6 = conv_block(u6, 512)

    u7 = tf.keras.layers.UpSampling2D(2)(c6)
    u7 = tf.keras.layers.concatenate([u7, c3])
    c7 = conv_block(u7, 256)

    u8 = tf.keras.layers.UpSampling2D(2)(c7)
    u8 = tf.keras.layers.concatenate([u8, c2])
    c8 = conv_block(u8, 128)

    u9 = tf.keras.layers.UpSampling2D(2)(c8)
    u9 = tf.keras.layers.concatenate([u9, c1])
    c9 = conv_block(u9, 64)

    # Output
    outputs = tf.keras.layers.Conv2D(num_classes, 1, activation='sigmoid')(c9)

    return tf.keras.Model(inputs, outputs)

def conv_block(inputs, filters):
    x = tf.keras.layers.Conv2D(filters, 3, activation='relu', padding='same')(inputs)
```

```
x = tf.keras.layers.Conv2D(filters, 3, activation='relu', padding='same')(x)
return x
```

Key Features:

- **Symmetric:** Encoder-decoder symmetry
- **Skip Connections:** Preserve spatial information
- **Data Efficient:** Works with limited training data

Applications:

- **Medical Imaging:** Cell segmentation, organ delineation
- **Satellite Imagery:** Land use classification
- **Materials Science:** Defect detection

Feature Detection & Matching

Q15: Compare SIFT, SURF, and ORB feature detectors.

Answer:

Feature Detection Algorithms:

Algorithm	Year	Key Properties	Speed	Rotation Invariant	Scale Invariant
SIFT	2004	High accuracy, patented	Slow	Yes	Yes
SURF	2006	Faster than SIFT	Medium	Yes	Yes
ORB	2011	Free, real-time	Fast	Yes	Limited

1. SIFT (Scale-Invariant Feature Transform):

```
import cv2

# SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and compute descriptors
keypoints, descriptors = sift.detectAndCompute(gray_image, None)

# Draw keypoints
img_with_keypoints = cv2.drawKeypoints(image, keypoints, None)

# Properties:
# - 128-dimensional descriptor
# - Highly distinctive
# - Computationally expensive
```

2. SURF (Speeded Up Robust Features):

```
# SURF detector (needs opencv-contrib-python)
surf = cv2.xfeatures2d.SURF_create(hessianThreshold=400)

keypoints, descriptors = surf.detectAndCompute(gray_image, None)

# Properties:
# - 64 or 128-dimensional descriptor
# - 3x faster than SIFT
# - Uses Hessian matrix for detection
```

3. ORB (Oriented FAST and Rotated BRIEF):

```
# ORB detector
orb = cv2.ORB_create(nfeatures=500)

keypoints, descriptors = orb.detectAndCompute(gray_image, None)

# Properties:
# - Binary descriptor (256 bits)
# - Very fast
# - Free to use (no patents)
# - Good for real-time applications
```

Feature Matching:

```
def match_features(desc1, desc2, algorithm='SIFT'):
    if algorithm in ['SIFT', 'SURF']:
        # FLANN matcher for float descriptors
        FLANN_INDEX_KDTREE = 1
        index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
        search_params = dict(checks=50)
        flann = cv2.FlannBasedMatcher(index_params, search_params)
        matches = flann.knnMatch(desc1, desc2, k=2)

    elif algorithm == 'ORB':
        # BruteForce matcher for binary descriptors
        bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
        matches = bf.match(desc1, desc2)
        matches = sorted(matches, key=lambda x: x.distance)

    return matches

# Lowe's ratio test for good matches
def filter_matches(matches, ratio_threshold=0.75):
    good_matches = []
    for m, n in matches:
        if m.distance <= ratio_threshold * n.distance:
            good_matches.append(m)
    return good_matches
```

When to Use:

- **SIFT:** High accuracy needed, computational resources available
- **SURF:** Balance between speed and accuracy
- **ORB:** Real-time applications, mobile devices

Q16: What is homography and how is it used in computer vision?

Answer:

Homography: Mathematical transformation that maps points from one plane to another in 3D space.

Mathematical Definition:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Where $(x'/w', y'/w')$ are the transformed coordinates

Types of Transformations:

1. **Translation:** Moving image
2. **Rotation:** Rotating image
3. **Scale:** Resizing image
4. **Shear:** Skewing image
5. **Perspective:** Projective transformation

Computing Homography:

```
def find_homography_ransac(src_pts, dst_pts, threshold=5.0, max_iterations=1000):
    """
    Find homography using RANSAC algorithm
    """
    best_homography = None
    best_inliers = 0

    for _ in range(max_iterations):
        # Randomly sample 4 point pairs
        sample_indices = np.random.choice(len(src_pts), 4, replace=False)
        sample_src = src_pts[sample_indices]
        sample_dst = dst_pts[sample_indices]

        # Compute homography from 4 points
        H = compute_homography_4_points(sample_src, sample_dst)

        if H is None:
            continue

        # Count inliers
        inliers = count_inliers(src_pts, dst_pts, H, threshold)
```

```

        if inliers > best_inliers:
            best_inliers = inliers
            best_homography = H

    return best_homography

# OpenCV implementation
H, mask = cv2.findHomography(src_points, dst_points,
                             cv2.RANSAC,
                             ransacReprojThreshold=5.0)

```

Applications:

1. Image Stitching:

```

def stitch_images(img1, img2):
    # Find keypoints and match
    sift = cv2.SIFT_create()
    kp1, desc1 = sift.detectAndCompute(img1, None)
    kp2, desc2 = sift.detectAndCompute(img2, None)

    # Match features
    matches = match_features(desc1, desc2)

    # Extract matched points
    src_pts = np.float32([kp1[m.queryIdx].pt for m in matches]).reshape(-1, 1, 2)
    dst_pts = np.float32([kp2[m.trainIdx].pt for m in matches]).reshape(-1, 1, 2)

    # Find homography
    H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC)

    # Warp and blend images
    warped = cv2.warpPerspective(img1, H, (img1.shape[1] + img2.shape[1], img1.shape[0]))
    warped[0:img2.shape[0], 0:img2.shape[1]] = img2

    return warped

```

2. Perspective Correction:

```

def correct_perspective(image, corners):
    """
    Correct perspective distortion given 4 corner points
    """
    # Define target rectangle
    width, height = 400, 300
    target_corners = np.float32([
        [0, 0], [width, 0], [width, height], [0, height]
    ])

    # Compute homography
    H = cv2.getPerspectiveTransform(corners, target_corners)

    # Apply transformation

```

```
corrected = cv2.warpPerspective(image, H, (width, height))

return corrected
```

3. Object Tracking:

- Track planar objects across frames
- Augmented reality applications
- Camera calibration

Transfer Learning in CV

Q17: Explain transfer learning in computer vision and its strategies.

Answer:

Transfer Learning: Using pre-trained models on large datasets (ImageNet) for new tasks with limited data.

Why Transfer Learning Works:

- **Feature Hierarchy:** Early layers learn general features (edges, textures)
- **Later Layers:** Learn task-specific features
- **Data Efficiency:** Requires less training data

Strategies:

1. Feature Extraction (Frozen Features):

```
# Load pre-trained model without top layer
base_model = tf.keras.applications.VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)

# Freeze base model weights
base_model.trainable = False

# Add custom classifier
model = tf.keras.Sequential([
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(num_classes, activation='softmax')
])

# Compile with higher learning rate
model.compile(
```

```
optimizer=tf.keras.optimizers.Adam(lr=0.001),
loss='categorical_crossentropy',
metrics=['accuracy']
)
```

2. Fine-tuning (Gradual Unfreezing):

```
# Step 1: Train classifier with frozen base
# ... (train model as above)

# Step 2: Unfreeze top layers of base model
base_model.trainable = True

# Fine-tune from this layer onwards
fine_tune_at = 100

# Freeze all layers before fine_tune_at
for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False

# Use lower learning rate for fine-tuning
model.compile(
    optimizer=tf.keras.optimizers.Adam(lr=0.0001), # Lower LR
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Continue training
model.fit(train_data, epochs=10, validation_data=val_data)
```

3. Progressive Resizing:

```
def progressive_training(model, train_data, val_data):
    sizes = [128, 192, 224] # Gradually increase image size

    for size in sizes:
        # Resize dataset
        train_resized = resize_dataset(train_data, size)
        val_resized = resize_dataset(val_data, size)

        # Train for few epochs at this resolution
        model.fit(train_resized,
                  validation_data=val_resized,
                  epochs=5)

        # Reduce learning rate
        model.optimizer.learning_rate *= 0.5
```

Domain-Specific Transfer Learning:

Medical Imaging:

```
# For medical images, ImageNet features may not be optimal
# Use models pre-trained on medical data

# CheXNet for chest X-rays
base_model = load_chexnet_weights()

# RadImageNet for general medical imaging
base_model = tf.keras.applications.ResNet50(
    weights='radimagenet', # Instead of 'imagenet'
    include_top=False
)
```

Best Practices:

- **Similar Domain:** More layers can be fine-tuned
- **Different Domain:** Use as feature extractor
- **Small Dataset:** Freeze more layers
- **Large Dataset:** Fine-tune more layers

Generative Models (GANs, VAEs)

Q18: Explain Generative Adversarial Networks (GANs) and their applications in computer vision.

Answer:

GANs: Two neural networks competing against each other - Generator creates fake data, Discriminator tries to detect fake from real.

Architecture:

```
class Generator(tf.keras.Model):
    def __init__(self, latent_dim=100):
        super().__init__()
        self.latent_dim = latent_dim

        self.model = tf.keras.Sequential([
            tf.keras.layers.Dense(256, activation='relu'),
            tf.keras.layers.BatchNormalization(),
            tf.keras.layers.Dense(512, activation='relu'),
            tf.keras.layers.BatchNormalization(),
            tf.keras.layers.Dense(1024, activation='relu'),
            tf.keras.layers.BatchNormalization(),
            tf.keras.layers.Dense(28*28, activation='tanh'),
            tf.keras.layers.Reshape((28, 28, 1))
        ])

    def call(self, noise):
        return self.model(noise)
```

```

class Discriminator(tf.keras.Model):
    def __init__(self):
        super().__init__()

        self.model = tf.keras.Sequential([
            tf.keras.layers.Flatten(),
            tf.keras.layers.Dense(1024, activation='relu'),
            tf.keras.layers.Dropout(0.3),
            tf.keras.layers.Dense(512, activation='relu'),
            tf.keras.layers.Dropout(0.3),
            tf.keras.layers.Dense(256, activation='relu'),
            tf.keras.layers.Dropout(0.3),
            tf.keras.layers.Dense(1, activation='sigmoid')
        ])

    def call(self, images):
        return self.model(images)

```

Training Process:

```

class GAN:
    def __init__(self, generator, discriminator):
        self.generator = generator
        self.discriminator = discriminator

        self.g_optimizer = tf.keras.optimizers.Adam(2e-4, beta_1=0.5)
        self.d_optimizer = tf.keras.optimizers.Adam(2e-4, beta_1=0.5)

    @tf.function
    def train_step(self, real_images, batch_size):
        # Generate random noise
        noise = tf.random.normal([batch_size, self.generator.latent_dim])

        with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
            # Generate fake images
            fake_images = self.generator(noise, training=True)

            # Discriminator predictions
            real_output = self.discriminator(real_images, training=True)
            fake_output = self.discriminator(fake_images, training=True)

            # Calculate losses
            gen_loss = self.generator_loss(fake_output)
            disc_loss = self.discriminator_loss(real_output, fake_output)

            # Apply gradients
            gen_gradients = gen_tape.gradient(gen_loss, self.generator.trainable_variables)
            disc_gradients = disc_tape.gradient(disc_loss, self.discriminator.trainable_variables)

            self.g_optimizer.apply_gradients(zip(gen_gradients, self.generator.trainable_variables))
            self.d_optimizer.apply_gradients(zip(disc_gradients, self.discriminator.trainable_variables))

        return gen_loss, disc_loss

    def generator_loss(self, fake_output):

```

```

# Generator wants discriminator to classify fake images as real
return tf.keras.losses.BinaryCrossentropy()(tf.ones_like(fake_output), fake_output)

def discriminator_loss(self, real_output, fake_output):
    # Discriminator wants to classify real as 1, fake as 0
    real_loss = tf.keras.losses.BinaryCrossentropy()(tf.ones_like(real_output), real_output)
    fake_loss = tf.keras.losses.BinaryCrossentropy()(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

```

GAN Variants:

1. DCGAN (Deep Convolutional GAN):

```

# Generator with transposed convolutions
def build_dcgan_generator():
    model = tf.keras.Sequential([
        tf.keras.layers.Dense(7*7*256, use_bias=False, input_shape=(100,)),
        tf.keras.layers.BatchNormalization(),
        tf.keras.layers.LeakyReLU(),
        tf.keras.layers.Reshape((7, 7, 256)),

        tf.keras.layers.Conv2DTranspose(128, (5, 5), strides=(1, 1), padding='same'),
        tf.keras.layers.BatchNormalization(),
        tf.keras.layers.LeakyReLU(),

        tf.keras.layers.Conv2DTranspose(64, (5, 5), strides=(2, 2), padding='same'),
        tf.keras.layers.BatchNormalization(),
        tf.keras.layers.LeakyReLU(),

        tf.keras.layers.Conv2DTranspose(1, (5, 5), strides=(2, 2), padding='same', activation='tanh')
    ])
    return model

```

2. Conditional GAN:

```

# Generate images conditioned on class labels
def build_conditional_generator(num_classes, latent_dim):
    noise = tf.keras.Input(shape=(latent_dim,))
    label = tf.keras.Input(shape=(1,))

    # Embed label
    label_embedding = tf.keras.layers.Embedding(num_classes, 50)(label)
    label_embedding = tf.keras.layers.Flatten()(label_embedding)

    # Concatenate noise and label
    input_layer = tf.keras.layers.Concatenate()([noise, label_embedding])

    # Generator layers...
    x = tf.keras.layers.Dense(256, activation='relu')(input_layer)
    # ... rest of generator

    return tf.keras.Model([noise, label], output)

```

Applications:

- **Data Augmentation:** Generate training samples
- **Super Resolution:** Enhance image quality
- **Style Transfer:** Change artistic style of images
- **Image Inpainting:** Fill missing parts of images
- **Domain Adaptation:** Convert between domains (day → night)

OpenCV & Implementation

Q19: How do you implement real-time object detection using OpenCV?

Answer:

Real-time Object Detection Pipeline:

1. Setup and Model Loading:

```
import cv2
import numpy as np

class RealTimeDetector:
    def __init__(self, model_path, config_path, classes_path):
        # Load YOLO model
        self.net = cv2.dnn.readNet(model_path, config_path)

        # Load class names
        with open(classes_path, 'r') as f:
            self.classes = [line.strip() for line in f.readlines()]

        # Get output layer names
        layer_names = self.net.getLayerNames()
        self.output_layers = [layer_names[i[0] - 1] for i in self.net.getUnconnectedOutLayers()]

        # Generate random colors for classes
        self.colors = np.random.uniform(0, 255, size=(len(self.classes), 3))

    def detect_objects(self, frame):
        height, width, channels = frame.shape

        # Prepare input blob
        blob = cv2.dnn.blobFromImage(frame, 0.00392, (416, 416), (0, 0, 0), True, crop=False)
        self.net.setInput(blob)

        # Run inference
        outputs = self.net.forward(self.output_layers)

        # Process outputs
        boxes, confidences, class_ids = self.process_outputs(outputs, width, height)

        # Apply Non-Maximum Suppression
```

```

        indices = cv2.dnn.NMSBoxes(bboxes, confidences, 0.5, 0.4)

        return self.draw_boxes(frame, bboxes, confidences, class_ids, indices)

def process_outputs(self, outputs, width, height):
    bboxes = []
    confidences = []
    class_ids = []

    for output in outputs:
        for detection in output:
            scores = detection[5:]
            class_id = np.argmax(scores)
            confidence = scores[class_id]

            if confidence > 0.5:
                # Object detected
                center_x = int(detection[0] * width)
                center_y = int(detection[1] * height)
                w = int(detection[2] * width)
                h = int(detection[3] * height)

                # Rectangle coordinates
                x = int(center_x - w / 2)
                y = int(center_y - h / 2)

                bboxes.append([x, y, w, h])
                confidences.append(float(confidence))
                class_ids.append(class_id)

    return bboxes, confidences, class_ids

def draw_boxes(self, frame, bboxes, confidences, class_ids, indices):
    if len(indices) > 0:
        for i in indices.flatten():
            x, y, w, h = bboxes[i]
            label = f"{self.classes[class_ids[i]]}: {confidences[i]:.2f}"
            color = self.colors[class_ids[i]]

            # Draw bounding box
            cv2.rectangle(frame, (x, y), (x + w, y + h), color, 2)

            # Draw label
            cv2.putText(frame, label, (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, color)

    return frame

# Real-time detection from webcam
def main():
    detector = RealTimeDetector('yolo.weights', 'yolo.cfg', 'coco.names')

    # Initialize webcam
    cap = cv2.VideoCapture(0)

    # FPS calculation
    fps_counter = 0

```

```

start_time = time.time()

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Detect objects
    result_frame = detector.detect_objects(frame)

    # Calculate and display FPS
    fps_counter += 1
    elapsed_time = time.time() - start_time
    if elapsed_time >= 1.0:
        fps = fps_counter / elapsed_time
        fps_counter = 0
        start_time = time.time()

    cv2.putText(result_frame, f"FPS: {fps:.1f}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

    # Display result
    cv2.imshow('Real-time Detection', result_frame)

    # Exit on 'q' key
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

if __name__ == "__main__":
    main()

```

2. Performance Optimization:

```

class OptimizedDetector:
    def __init__(self, model_path):
        # Use GPU if available
        self.net = cv2.dnn.readNet(model_path)

        # Set backend and target
        if cv2.cuda.getCudaEnabledDeviceCount() > 0:
            self.net.setPreferableBackend(cv2.dnn.DNN_BACKEND_CUDA)
            self.net.setPreferableTarget(cv2.dnn.DNN_TARGET_CUDA)
        else:
            self.net.setPreferableBackend(cv2.dnn.DNN_BACKEND_OPENCV)
            self.net.setPreferableTarget(cv2.dnn.DNN_TARGET_CPU)

    def detect_with_threading(self, frame_queue, result_queue):
        """Process frames in separate thread"""
        while True:
            if not frame_queue.empty():
                frame = frame_queue.get()

```

```
result = self.detect_objects(frame)
result_queue.put(result)
```

3. Multi-object Tracking:

```
class MultiObjectTracker:
    def __init__(self):
        self.trackers = []
        self.next_id = 0

    def update(self, detections):
        # Update existing trackers
        for tracker in self.trackers:
            success, bbox = tracker['tracker'].update(frame)
            if success:
                tracker['bbox'] = bbox
            else:
                self.trackers.remove(tracker)

        # Assign new detections to trackers
        for detection in detections:
            # Find best matching tracker or create new one
            matched_tracker = self.find_matching_tracker(detection)

            if matched_tracker is None:
                # Create new tracker
                new_tracker = cv2.TrackerCSRT_create()
                new_tracker.init(frame, tuple(detection))

                self.trackers.append({
                    'id': self.next_id,
                    'tracker': new_tracker,
                    'bbox': detection
                })
                self.next_id += 1
```

Evaluation Metrics

Q20: What are the key evaluation metrics for object detection?

Answer:

Object Detection Metrics:

1. Intersection over Union (IoU):

```
def calculate_iou(box1, box2):
    """
    Calculate IoU between two bounding boxes
    Args:
        box1, box2: [x1, y1, x2, y2] format
```

```

"""
# Calculate intersection area
x1 = max(box1[0], box2[0])
y1 = max(box1[1], box2[1])
x2 = min(box1[2], box2[2])
y2 = min(box1[3], box2[3])

if x2 <= x1 or y2 <= y1:
    return 0.0

intersection = (x2 - x1) * (y2 - y1)

# Calculate union area
area1 = (box1[2] - box1[0]) * (box1[3] - box1[1])
area2 = (box2[2] - box2[0]) * (box2[3] - box2[1])
union = area1 + area2 - intersection

return intersection / union if union > 0 else 0.0

```

2. Precision and Recall:

```

def calculate_precision_recall(predictions, ground_truths, iou_threshold=0.5):
    """
    Calculate precision and recall for object detection
    """
    tp = 0 # True positives
    fp = 0 # False positives
    fn = 0 # False negatives

    # Match predictions to ground truths
    matched_gt = set()

    for pred in predictions:
        best_iou = 0
        best_gt_idx = -1

        for gt_idx, gt in enumerate(ground_truths):
            if gt_idx in matched_gt:
                continue

            iou = calculate_iou(pred['bbox'], gt['bbox'])

            if iou > best_iou and pred['class'] == gt['class']:
                best_iou = iou
                best_gt_idx = gt_idx

        if best_iou >= iou_threshold:
            tp += 1
            matched_gt.add(best_gt_idx)
        else:
            fp += 1

    fn = len(ground_truths) - len(matched_gt)

    precision = tp / (tp + fp) if (tp + fp) > 0 else 0

```

```

recall = tp / (tp + fn) if (tp + fn) > 0 else 0

return precision, recall

```

3. Average Precision (AP):

```

def calculate_average_precision(predictions, ground_truths, iou_threshold=0.5):
    """
    Calculate Average Precision using 11-point interpolation
    """
    # Sort predictions by confidence
    predictions = sorted(predictions, key=lambda x: x['confidence'], reverse=True)

    precisions = []
    recalls = []

    for i in range(1, len(predictions) + 1):
        current_preds = predictions[:i]
        precision, recall = calculate_precision_recall(current_preds, ground_truths, iou_threshold)
        precisions.append(precision)
        recalls.append(recall)

    # Calculate AP using 11-point interpolation
    ap = 0
    for recall_threshold in np.arange(0, 1.1, 0.1):
        # Find precision at this recall level
        valid_precisions = [p for p, r in zip(precisions, recalls) if r >= recall_threshold]
        max_precision = max(valid_precisions) if valid_precisions else 0
        ap += max_precision

    return ap / 11 # Average over 11 points

```

4. Mean Average Precision (mAP):

```

def calculate_map(predictions_per_class, ground_truths_per_class, iou_thresholds=[0.5]):
    """
    Calculate mAP across classes and IoU thresholds
    """
    aps = []

    for class_name in ground_truths_per_class.keys():
        class_preds = predictions_per_class.get(class_name, [])
        class_gts = ground_truths_per_class[class_name]

        class_aps = []
        for iou_thresh in iou_thresholds:
            ap = calculate_average_precision(class_preds, class_gts, iou_thresh)
            class_aps.append(ap)

        aps.append(np.mean(class_aps))

    return np.mean(aps)

# COCO-style mAP (IoU from 0.5 to 0.95)

```

```
def calculate_coco_map(predictions, ground_truths):
    iou_thresholds = np.arange(0.5, 1.0, 0.05)
    return calculate_map(predictions, ground_truths, iou_thresholds)
```

5. Detection Quality Assessment:

```
class DetectionEvaluator:
    def __init__(self, classes, iou_threshold=0.5):
        self.classes = classes
        self.iou_threshold = iou_threshold
        self.reset()

    def reset(self):
        self.all_predictions = {cls: [] for cls in self.classes}
        self.all_ground_truths = {cls: [] for cls in self.classes}

    def add_batch(self, predictions, ground_truths):
        for pred in predictions:
            self.all_predictions[pred['class']].append(pred)

        for gt in ground_truths:
            self.all_ground_truths[gt['class']].append(gt)

    def compute_metrics(self):
        results = {}

        for class_name in self.classes:
            preds = self.all_predictions[class_name]
            gts = self.all_ground_truths[class_name]

            if len(gts) == 0:
                continue

            ap = calculate_average_precision(preds, gts, self.iou_threshold)
            precision, recall = calculate_precision_recall(preds, gts, self.iou_threshold)

            results[class_name] = {
                'AP': ap,
                'Precision': precision,
                'Recall': recall,
                'F1': 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0
            }

        # Overall metrics
        results['mAP'] = np.mean([r['AP'] for r in results.values() if 'AP' in r])

        return results
```

Metric Interpretation:

- **IoU > 0.5:** Generally considered good detection
- **mAP@0.5:** Standard object detection metric
- **mAP@0.5:0.95:** COCO evaluation (more stringent)

- **Precision:** How many detections are correct
- **Recall:** How many objects are found

Advanced Topics

Q21: Explain attention mechanisms in computer vision.

Answer:

Attention Mechanisms: Allow models to focus on relevant parts of input when making predictions.

Types in Computer Vision:

1. Spatial Attention:

```
class SpatialAttention(tf.keras.layers.Layer):
    def __init__(self, filters):
        super().__init__()
        self.conv1 = tf.keras.layers.Conv2D(filters, 1, activation='relu')
        self.conv2 = tf.keras.layers.Conv2D(1, 1, activation='sigmoid')

    def call(self, x):
        # Generate attention map
        attention = self.conv1(x)
        attention = self.conv2(attention)

        # Apply attention to input
        attended = x * attention

        return attended, attention

# Usage in model
class AttentionNet(tf.keras.Model):
    def __init__(self, num_classes):
        super().__init__()
        self.backbone = tf.keras.applications.ResNet50(include_top=False)
        self.attention = SpatialAttention(512)
        self.classifier = tf.keras.layers.Dense(num_classes, activation='softmax')

    def call(self, x):
        features = self.backbone(x)
        attended_features, attention_map = self.attention(features)

        # Global average pooling
        pooled = tf.keras.layers.GlobalAveragePooling2D()(attended_features)
        output = self.classifier(pooled)

        return output, attention_map
```

2. Channel Attention (Squeeze-and-Excitation):

```

class SEBlock(tf.keras.layers.Layer):
    def __init__(self, filters, reduction=16):
        super().__init__()
        self.global_avg_pool = tf.keras.layers.GlobalAveragePooling2D()
        self.fc1 = tf.keras.layers.Dense(filters // reduction, activation='relu')
        self.fc2 = tf.keras.layers.Dense(filters, activation='sigmoid')
        self.reshape = tf.keras.layers.Reshape((1, 1, filters))

    def call(self, x):
        # Squeeze: Global average pooling
        squeeze = self.global_avg_pool(x)

        # Excitation: Two FC layers
        excitation = self.fc1(squeeze)
        excitation = self.fc2(excitation)
        excitation = self.reshape(excitation)

        # Scale feature maps
        return x * excitation

```

3. Self-Attention (Non-local Networks):

```

class SelfAttention(tf.keras.layers.Layer):
    def __init__(self, filters):
        super().__init__()
        self.filters = filters
        self.query_conv = tf.keras.layers.Conv2D(filters // 8, 1)
        self.key_conv = tf.keras.layers.Conv2D(filters // 8, 1)
        self.value_conv = tf.keras.layers.Conv2D(filters, 1)
        self.output_conv = tf.keras.layers.Conv2D(filters, 1)
        self.softmax = tf.keras.layers.Softmax(axis=-1)

    def call(self, x):
        batch_size, height, width, channels = x.shape

        # Generate query, key, value
        query = self.query_conv(x)
        key = self.key_conv(x)
        value = self.value_conv(x)

        # Reshape for matrix operations
        query = tf.reshape(query, (batch_size, -1, channels // 8))
        key = tf.reshape(key, (batch_size, -1, channels // 8))
        value = tf.reshape(value, (batch_size, -1, channels))

        # Compute attention
        attention = tf.matmul(query, key, transpose_b=True)
        attention = self.softmax(attention)

        # Apply attention to values
        out = tf.matmul(attention, value)
        out = tf.reshape(out, (batch_size, height, width, channels))

        # Output projection

```

```

out = self.output_conv(out)

# Residual connection
return out + x

```

4. Vision Transformer (ViT):

```

class VisionTransformer(tf.keras.Model):
    def __init__(self, image_size, patch_size, num_classes, d_model, num_heads, num_layers):
        super().__init__()
        self.patch_size = patch_size
        self.num_patches = (image_size // patch_size) ** 2
        self.d_model = d_model

        # Patch embedding
        self.patch_embed = tf.keras.layers.Dense(d_model)

        # Position embedding
        self.pos_embed = self.add_weight(
            shape=(1, self.num_patches + 1, d_model),
            initializer='random_normal'
        )

        # Class token
        self.cls_token = self.add_weight(
            shape=(1, 1, d_model),
            initializer='random_normal'
        )

        # Transformer blocks
        self.transformer_blocks = [
            TransformerBlock(d_model, num_heads) for _ in range(num_layers)
        ]

        # Classification head
        self.classifier = tf.keras.layers.Dense(num_classes, activation='softmax')

    def call(self, x):
        batch_size = tf.shape(x)[0]

        # Extract patches
        patches = self.extract_patches(x)

        # Embed patches
        embedded = self.patch_embed(patches)

        # Add class token
        cls_tokens = tf.tile(self.cls_token, [batch_size, 1, 1])
        embedded = tf.concat([cls_tokens, embedded], axis=1)

        # Add position embedding
        embedded += self.pos_embed

        # Apply transformer blocks
        for block in self.transformer_blocks:

```

```

        embedded = block(embedded)

    # Use class token for classification
    cls_output = embedded[:, 0]
    return self.classifier(cls_output)

```

Applications:

- **Object Detection:** Focus on relevant regions
- **Image Captioning:** Attend to different image parts when generating words
- **Visual Question Answering:** Attend to question-relevant image regions

Practical Applications

Q22: How do you implement a complete face recognition system?

Answer:

Face Recognition Pipeline:

1. Face Detection:

```

import cv2
import numpy as np
from deepface import DeepFace
import os
import pickle

class FaceRecognitionSystem:
    def __init__(self, database_path="face_database"):
        self.database_path = database_path
        self.face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
        self.known_encodings = {}
        self.load_database()

    def detect_faces(self, image):
        """Detect faces in image using Haar cascades"""
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        faces = self.face_cascade.detectMultiScale(
            gray,
            scaleFactor=1.1,
            minNeighbors=5,
            minSize=(30, 30)
        )
        return faces

    def extract_face_embedding(self, image, face_coords):
        """Extract face embedding using DeepFace"""
        x, y, w, h = face_coords
        face_image = image[y:y+h, x:x+w]

        try:

```

```

        # Use DeepFace for embedding extraction
        embedding = DeepFace.represent(
            face_image,
            model_name='Facenet',
            enforce_detection=False
        )
        return np.array(embedding[0]['embedding'])
    except:
        return None

def add_person_to_database(self, person_name, image_paths):
    """Add a new person to the face database"""
    person_encodings = []

    for image_path in image_paths:
        image = cv2.imread(image_path)
        faces = self.detect_faces(image)

        for face in faces:
            embedding = self.extract_face_embedding(image, face)
            if embedding is not None:
                person_encodings.append(embedding)

    if person_encodings:
        # Average embeddings for this person
        avg_encoding = np.mean(person_encodings, axis=0)
        self.known_encodings[person_name] = avg_encoding
        self.save_database()
        print(f"Added {person_name} to database with {len(person_encodings)} face samples")

def recognize_face(self, face_embedding, threshold=0.6):
    """Recognize face from embedding"""
    if not self.known_encodings:
        return "Unknown", 0.0

    min_distance = float('inf')
    recognized_person = "Unknown"

    for person_name, known_embedding in self.known_encodings.items():
        # Calculate Euclidean distance
        distance = np.linalg.norm(face_embedding - known_embedding)

        if distance < min_distance:
            min_distance = distance
            recognized_person = person_name

    # Check if distance is within threshold
    confidence = max(0, 1 - min_distance)
    if min_distance <= threshold:
        return recognized_person, confidence
    else:
        return "Unknown", confidence

def process_frame(self, frame):
    """Process a single frame for face recognition"""
    faces = self.detect_faces(frame)

```

```

results = []

for (x, y, w, h) in faces:
    # Extract embedding
    embedding = self.extract_face_embedding(frame, (x, y, w, h))

    if embedding is not None:
        # Recognize face
        person_name, confidence = self.recognize_face(embedding)

        # Draw bounding box and label
        color = (0, 255, 0) if person_name != "Unknown" else (0, 0, 255)
        cv2.rectangle(frame, (x, y), (x+w, y+h), color, 2)

        label = f"{person_name}: {confidence:.2f}"
        cv2.putText(frame, label, (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.8, color)

        results.append({
            'name': person_name,
            'confidence': confidence,
            'bbox': (x, y, w, h)
        })

return frame, results

def save_database(self):
    """Save face encodings to disk"""
    os.makedirs(self.database_path, exist_ok=True)
    with open(os.path.join(self.database_path, 'encodings.pkl'), 'wb') as f:
        pickle.dump(self.known_encodings, f)

def load_database(self):
    """Load face encodings from disk"""
    encoding_file = os.path.join(self.database_path, 'encodings.pkl')
    if os.path.exists(encoding_file):
        with open(encoding_file, 'rb') as f:
            self.known_encodings = pickle.load(f)
        print(f"Loaded {len(self.known_encodings)} persons from database")

# Real-time face recognition
def main():
    system = FaceRecognitionSystem()

    # Add persons to database (run once)
    # system.add_person_to_database("John", ["john1.jpg", "john2.jpg", "john3.jpg"])
    # system.add_person_to_database("Jane", ["jane1.jpg", "jane2.jpg", "jane3.jpg"])

    # Start real-time recognition
    cap = cv2.VideoCapture(0)

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        # Process frame

```

```

        processed_frame, results = system.process_frame(frame)

        # Display results
        cv2.imshow('Face Recognition', processed_frame)

        # Print recognition results
        for result in results:
            print(f"Detected: {result['name']} (Confidence: {result['confidence']:.2f})")

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()

```

2. Advanced Features:

```

class AdvancedFaceSystem(FaceRecognitionSystem):
    def __init__(self):
        super().__init__()
        self.face_tracker = {} # Track faces across frames
        self.emotion_model = self.load_emotion_model()

    def detect_emotions(self, face_image):
        """Detect emotions in face image"""
        try:
            result = DeepFace.analyze(
                face_image,
                actions=['emotion'],
                enforce_detection=False
            )
            return result[0]['emotion']
        except:
            return None

    def estimate_age_gender(self, face_image):
        """Estimate age and gender"""
        try:
            result = DeepFace.analyze(
                face_image,
                actions=['age', 'gender'],
                enforce_detection=False
            )
            return result[0]['age'], result[0]['gender']
        except:
            return None, None

    def anti_spoofing_check(self, face_image):
        """Simple liveness detection"""
        # Convert to grayscale
        gray = cv2.cvtColor(face_image, cv2.COLOR_BGR2GRAY)

```

```

# Calculate variance of Laplacian (blur detection)
laplacian_var = cv2.Laplacian(gray, cv2.CV_64F).var()

# Simple threshold for liveness (not production-ready)
is_live = laplacian_var > 100

return is_live, laplacian_var

```

Performance Optimization:

- Use face alignment before embedding extraction
- Implement face quality assessment
- Cache embeddings for known faces
- Use GPU acceleration for deep learning models
- Implement multi-threading for real-time processing

Deployment & Optimization

Q23: How do you optimize computer vision models for deployment?

Answer:

Model Optimization Strategies:

1. Model Quantization:

```

import tensorflow as tf

# Post-training quantization
def quantize_model(model_path, output_path):
    # Convert to TensorFlow Lite with quantization
    converter = tf.lite.TFLiteConverter.from_saved_model(model_path)

    # Dynamic range quantization (weights only)
    converter.optimizations = [tf.lite.Optimize.DEFAULT]

    # Full integer quantization (requires representative dataset)
    def representative_dataset():
        for _ in range(100):
            # Generate representative data
            data = np.random.random((1, 224, 224, 3)).astype(np.float32)
            yield [data]

    converter.representative_dataset = representative_dataset
    converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
    converter.inference_input_type = tf.int8
    converter.inference_output_type = tf.int8

    tflite_model = converter.convert()

```

```

# Save quantized model
with open(output_path, 'wb') as f:
    f.write(tflite_model)

# PyTorch quantization
import torch.quantization as quantization

def quantize_pytorch_model(model, calibration_loader):
    # Prepare model for quantization
    model.eval()
    model.qconfig = quantization.get_default_qconfig('fbgemm')
    model_prepared = quantization.prepare(model)

    # Calibrate with representative data
    with torch.no_grad():
        for data, _ in calibration_loader:
            model_prepared(data)

    # Convert to quantized model
    model_quantized = quantization.convert(model_prepared)

    return model_quantized

```

2. Model Pruning:

```

import tensorflow_model_optimization as tfmot

def prune_model(model, target_sparsity=0.5):
    # Define pruning schedule
    pruning_schedule = tfmot.sparsity.keras.PolynomialDecay(
        initial_sparsity=0.0,
        final_sparsity=target_sparsity,
        begin_step=0,
        end_step=1000
    )

    # Apply pruning to model
    pruned_model = tfmot.sparsity.keras.prune_low_magnitude(
        model,
        pruning_schedule=pruning_schedule
    )

    # Compile and train
    pruned_model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return pruned_model

# PyTorch pruning
import torch.nn.utils.prune as prune

def prune_pytorch_model(model, amount=0.3):

```

```

for name, module in model.named_modules():
    if isinstance(module, torch.nn.Conv2d):
        prune.l1_unstructured(module, name='weight', amount=amount)
    elif isinstance(module, torch.nn.Linear):
        prune.l1_unstructured(module, name='weight', amount=amount)

# Remove pruning reparameterization
for name, module in model.named_modules():
    if isinstance(module, (torch.nn.Conv2d, torch.nn.Linear)):
        prune.remove(module, 'weight')

return model

```

3. Knowledge Distillation:

```

class DistillationTrainer:
    def __init__(self, teacher_model, student_model, temperature=3.0, alpha=0.7):
        self.teacher = teacher_model
        self.student = student_model
        self.temperature = temperature
        self.alpha = alpha

        self.teacher.eval() # Teacher in eval mode

    def distillation_loss(self, student_logits, teacher_logits, true_labels):
        # Soft targets from teacher
        soft_targets = tf.nn.softmax(teacher_logits / self.temperature)
        soft_predictions = tf.nn.softmax(student_logits / self.temperature)

        # Distillation loss (KL divergence)
        distill_loss = tf.keras.losses.KLDivergence()(soft_targets, soft_predictions)

        # Standard loss with true labels
        standard_loss = tf.keras.losses.SparseCategoricalCrossentropy()(
            true_labels, student_logits
        )

        # Combined loss
        total_loss = (self.alpha * distill_loss * (self.temperature ** 2) +
                      (1 - self.alpha) * standard_loss)

        return total_loss

    def train_step(self, x_batch, y_batch):
        with tf.GradientTape() as tape:
            # Get teacher predictions
            teacher_logits = self.teacher(x_batch, training=False)

            # Get student predictions
            student_logits = self.student(x_batch, training=True)

            # Calculate distillation loss
            loss = self.distillation_loss(student_logits, teacher_logits, y_batch)

        # Update student model

```

```

gradients = tape.gradient(loss, self.student.trainable_variables)
optimizer.apply_gradients(zip(gradients, self.student.trainable_variables))

return loss

```

4. Edge Deployment Optimization:

```

class EdgeOptimizer:
    def __init__(self, target_platform='cpu'):
        self.target_platform = target_platform

    def optimize_for_inference(self, model):
        # Convert to ONNX for cross-platform deployment
        dummy_input = torch.randn(1, 3, 224, 224)
        torch.onnx.export(
            model,
            dummy_input,
            "model.onnx",
            export_params=True,
            opset_version=11,
            do_constant_folding=True,
            input_names=['input'],
            output_names=['output']
        )

        # Optimize ONNX model
        import onnxruntime as ort

        # Create inference session
        sess_options = ort.SessionOptions()
        sess_options.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL

        session = ort.InferenceSession("model.onnx", sess_options)

        return session

    def benchmark_model(self, model, input_shape, num_runs=100):
        """Benchmark model inference time"""
        import time

        # Warm up
        dummy_input = torch.randn(input_shape)
        for _ in range(10):
            _ = model(dummy_input)

        # Benchmark
        start_time = time.time()
        for _ in range(num_runs):
            _ = model(dummy_input)
        end_time = time.time()

        avg_time = (end_time - start_time) / num_runs
        fps = 1.0 / avg_time

        return {

```

```

        'avg_inference_time': avg_time,
        'fps': fps,
        'total_time': end_time - start_time
    }

```

5. Mobile Deployment:

```

# TensorFlow Lite for mobile
def convert_to_mobile(model_path):
    converter = tf.lite.TFLiteConverter.from_saved_model(model_path)

    # Optimize for mobile
    converter.optimizations = [tf.lite.Optimize.DEFAULT]
    converter.target_spec.supported_types = [tf.float16]

    # Generate model
    tflite_model = converter.convert()

    # Save for mobile app
    with open('model_mobile.tflite', 'wb') as f:
        f.write(tflite_model)

# CoreML for iOS
def convert_to_coreml(model, input_shape):
    import coremltools as ct

    # Convert PyTorch model to CoreML
    traced_model = torch.jit.trace(model, torch.randn(input_shape))
    coreml_model = ct.convert(
        traced_model,
        inputs=[ct.ImageType(shape=input_shape)]
    )

    # Save CoreML model
    coreml_model.save('model.mlmodel')

```

Deployment Checklist:

- Model size and inference speed requirements
- Hardware constraints (CPU/GPU/memory)
- Accuracy vs efficiency tradeoff
- Real-time processing requirements
- Battery life considerations (mobile)
- Network connectivity (edge vs cloud)

Quick Reference Guide

Common OpenCV Operations

```
# Basic operations
img = cv2.imread('image.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
resized = cv2.resize(img, (224, 224))
blurred = cv2.GaussianBlur(img, (15, 15), 0)
edges = cv2.Canny(gray, 50, 150)

# Drawing
cv2.rectangle(img, (x, y), (x+w, y+h), (255, 0, 0), 2)
cv2.circle(img, (cx, cy), radius, (0, 255, 0), -1)
cv2.putText(img, 'Text', (x, y), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)
```

Model Architecture Patterns

```
# CNN Block Pattern
def conv_block(x, filters, kernel_size=3, stride=1):
    x = tf.keras.layers.Conv2D(filters, kernel_size, stride, padding='same')(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.ReLU()(x)
    return x

# Residual Block
def residual_block(x, filters):
    shortcut = x
    x = conv_block(x, filters)
    x = conv_block(x, filters, activation=None)
    x = tf.keras.layers.Add()([x, shortcut])
    x = tf.keras.layers.ReLU()(x)
    return x
```

Performance Optimization Tips

1. **Use appropriate input sizes** - Smaller inputs = faster inference
2. **Batch processing** - Process multiple images together
3. **Model quantization** - Reduce precision for speed
4. **GPU acceleration** - Use CUDA for training/inference
5. **Caching** - Store processed features when possible
6. **Multi-threading** - Parallel processing for real-time applications

Interview Tips

1. **Understand the problem** - Classification, detection, or segmentation?
2. **Consider constraints** - Real-time requirements, accuracy needs
3. **Know trade-offs** - Speed vs accuracy, model size vs performance
4. **Practical experience** - Discuss projects and challenges faced
5. **Stay updated** - Latest architectures (Vision Transformers, EfficientNet)
6. **Code readiness** - Be prepared to implement basic CV operations

This Computer Vision cheat sheet covers fundamental concepts through advanced applications. Practice implementing these concepts and experiment with different architectures and optimization techniques to build strong practical skills.